A Complete Beginner's Guide

Building StartupIQ

How a real, full-stack, AI-powered web app is built
— and the tools behind it, explained from zero.

FastAPI · Supabase · Redis · Docker · Kubernetes · GitHub Actions
A hands-on tour of backend systems, scaling, and shipping software.

What's inside

Read it top to bottom, or jump to the tool you want to learn. Every lesson explains the idea from scratch, shows a diagram, ties it back to the StartupIQ app, and tells you when to use it elsewhere.

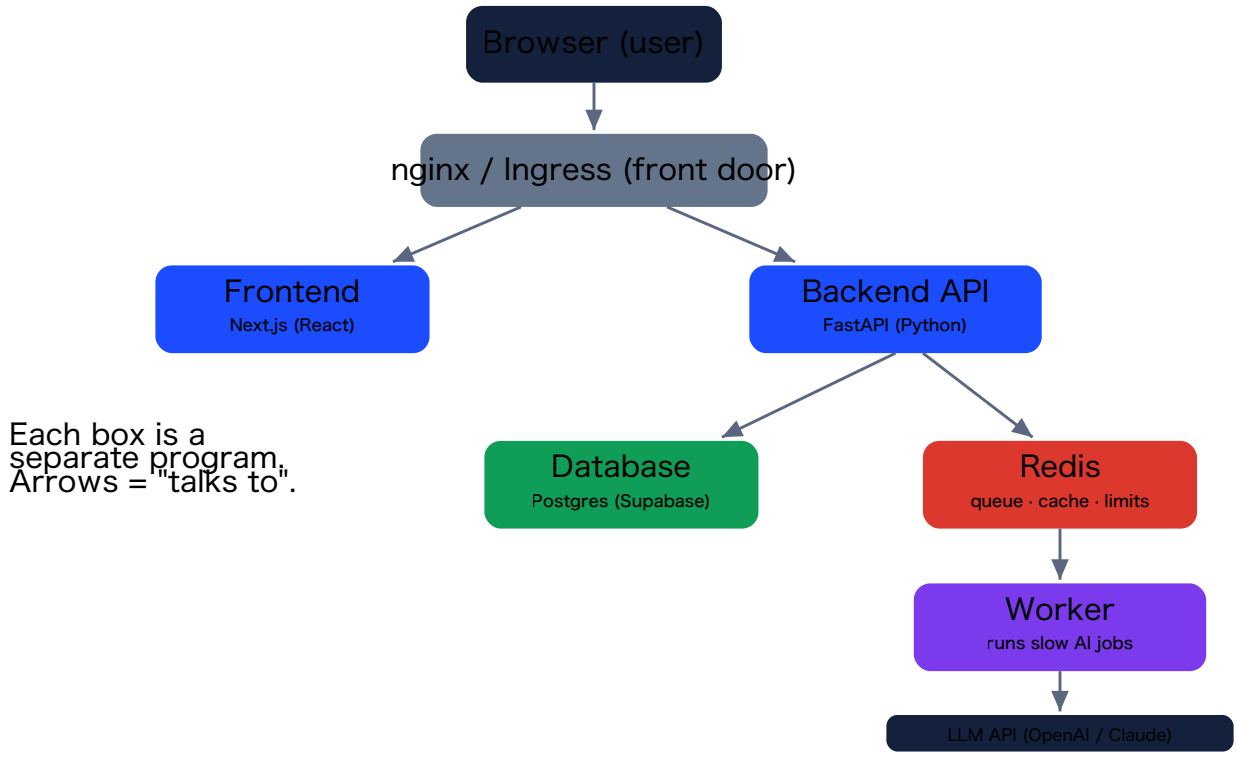
1 What We Built & The Big Picture

StartupIQ is a web app where you type in a startup idea and an AI evaluates it across six dimensions — competitors, target customers, market opportunity, risks, an MVP plan, and revenue models — and shows the results as visual tiles. But the app is really an excuse: building it end-to-end is how you learn the tools that turn a script into a real, scalable product.

This book teaches those tools from zero. You do not need prior experience with any of them. We will build up one idea at a time, and every lesson connects back to how StartupIQ actually uses it.

The cast of characters

A real web product is never one program. It is several specialised programs that talk to each other. Here is the whole system. Don't worry about the details yet — just get the shape.



The StartupIQ system. The browser only ever talks to the front door; everything else is hidden behind it.

One job each

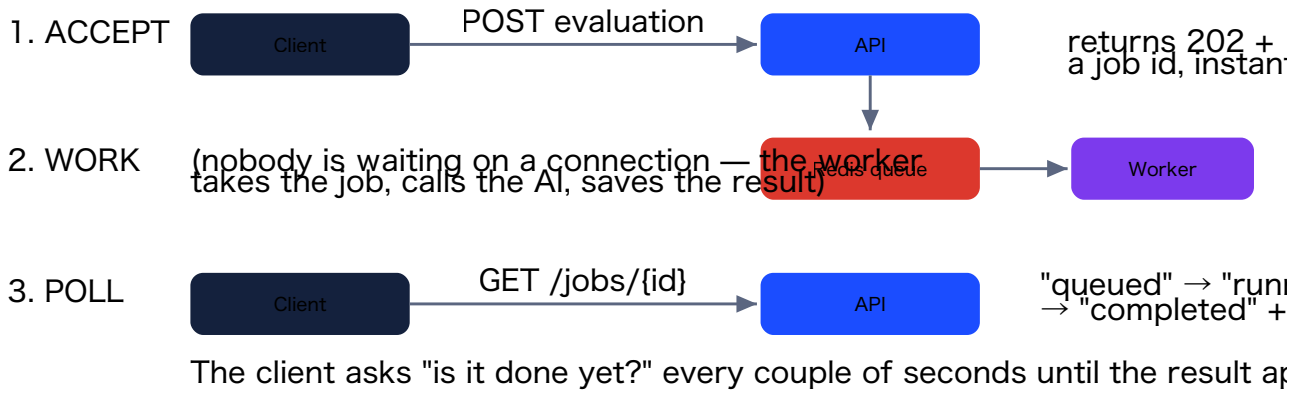
Piece	Its one job
Frontend (Next.js)	The face. What the user sees and clicks. Knows nothing about the database — it only calls the API.
Backend API (FastAPI)	The brain & traffic controller. Checks who you are, enforces rules, reads/writes the database, hands slow work to the worker.

Database (Postgres/ Supabase)	Permanent memory. Stores users, ideas, results.
Redis	A fast scratchpad that plays three roles: a job queue, a cache, and a rate-limit counter.
Worker (Arq)	Does the slow AI calls in the background so the API stays fast.
LLM API	The actual AI (OpenAI in development, Claude in production).

> **THE KEY IDEA: LAYERS**
>
> Notice nothing reaches across the diagram. The browser talks to the front door; the front door talks to the API; the API talks to the database. Each part has one clear job and only talks to its neighbours. This is called **separation of concerns**, and it is the single most important habit in building software that doesn't collapse as it grows. When you later add a feature, you touch *one* layer, not everything.

The flow that matters most: don't make users wait

Asking an AI to analyse an idea takes 10–20 seconds. If the API just sat there waiting, the user's browser would freeze and the server would get clogged. So StartupIQ uses a pattern you will see everywhere in real systems: **accept the request instantly, do the slow work in the background, and let the client check back for the result.**



The accept → work → poll pattern. The API answers in milliseconds; the slow part happens where no one is waiting.

That single pattern is why StartupIQ stays responsive, and it shows up again and again — "we'll email you when your export is ready" is the same idea. The rest of this book unpacks each tool that makes a system like this work, starting with the front door of the backend: the API.

2 APIs and FastAPI

The outside world — your frontend, a phone app, a `curl` command — can't touch the database directly (that would be a security nightmare). It goes through an **API**: a controlled set of doors, each with rules about who may enter and what they may do.

> **ANALOGY**
> An API is like the **counter at a restaurant**. You don't walk into the kitchen and cook. You place an order at the counter ("I'd like the pasta"), and the kitchen does the work and hands back a result. The counter is the only way in, and it follows rules. The API is your app's counter.

The vocabulary (this is most of it)

An API call is just an **HTTP request**. It has four parts:

- **Method** — the *verb*, what you want to do: `GET` (read), `POST` (create), `PATCH` (update), `DELETE` (remove).
- **URL / path** — the *noun*, which thing: `/api/v1/ideas/123`.
- **Body** — the data you send, as JSON (a simple text format of keys and values).
- **Status code** — the result, as a number: `200` OK, `201` Created, `404` Not Found, `401` Unauthorized. You'll memorise these without trying.

REST: design URLs around things, not actions

The popular style we use is **REST**. Its rule: build your URLs around *resources* (nouns) and use the HTTP method for the verb. So for "ideas" we don't invent `/createIdea` and `/getMyIdeas`. We have one noun, `/ideas`, and vary the method:

Request	Means
`POST /api/v1/ideas`	Create a new idea
`GET /api/v1/ideas`	List my ideas
`GET /api/v1/ideas/{id}`	Get one idea
`PATCH /api/v1/ideas/{id}`	Update an idea
`DELETE /api/v1/ideas/{id}`	Remove an idea

Once you learn this pattern, you can *guess* the API for any new thing. The `v1` in the path is **versioning**: someday you'll change the API in a way that would break old clients, so you'll add `/api/v2/` and leave `v1` alone. Building it in from day one is free insurance.

What FastAPI is

FastAPI is the Python framework we use to build the API. You write a normal Python function and decorate it to say "this function answers this URL." FastAPI handles reading the request, checking the data, and turning your return value into JSON.

```
@router.post("/ideas", status_code=201)        # POST /ideas → 201 Created
async def create_idea(
    payload: IdeaCreate,                        # the JSON body, auto-validated
    user_id: UUID = Depends(get_current_user),  # who is calling (see Ch.4)
    session: AsyncSession = Depends(get_session)# a database connection
):
    idea = StartupIdea(**payload.model_dump(), user_id=user_id)
    session.add(idea)
    await session.commit()                      # save it
    return idea                                 # FastAPI turns this into JSON
```

Two ideas worth pausing on

Validation for free. The `payload: IdeaCreate` line tells FastAPI "the body must look like an `IdeaCreate`." We define `IdeaCreate` with the fields and rules (e.g. title can't be empty), and FastAPI rejects bad input automatically *before your code runs*. We use a library called **Pydantic** for this — it's like a bouncer checking IDs at the door.

Dependency injection — the `Depends(...)` bits. Instead of every function manually doing "figure out who's calling, open a database connection," it just *declares what it needs*, and FastAPI supplies it. The function reads like a statement of requirements. This keeps shared logic (like checking the login) in one place instead of copy-pasted everywhere.

> **IN STARTUPIQ**
>
> Every protected endpoint has `user_id = Depends(get_current_user)`, and every database query filters by that user (`where user_id == current user`). The login proves *who you are* (authentication); this filter enforces *what you're allowed to see* (authorization). They are different checks, and you need both. A nice detail: if you ask for someone else's idea, we return `404 Not Found`, not `403 Forbidden` — saying "forbidden" would leak the fact that the idea exists.

> **WHEN YOU'LL USE THIS ELSEWHERE**
>
> Any time two programs need to talk over a network — a mobile app talking to a server, one microservice calling another, a third party integrating with you — you build (or call) a REST API. FastAPI (Python), Express (Node.js), and Spring (Java) are all popular ways to build one. The concepts (methods, paths, status codes, JSON, validation) are identical across all of them.

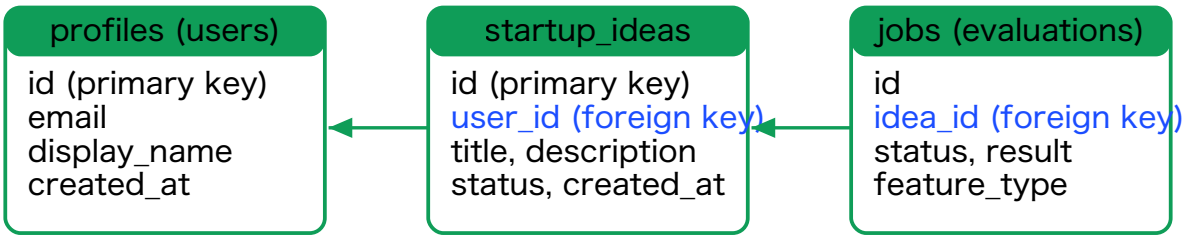
FastAPI also gives you a free, clickable API explorer at `/docs` — open it in a browser and you can try every endpoint without writing any code. Great for testing.

3 Databases (Postgres, Indexes, Migrations)

A web app without a database is a goldfish: it forgets everything the moment it restarts. The database is the app's permanent memory.

Why a "relational" database

Our data is full of relationships: a user *has many* ideas; an idea *has many* evaluations. When data looks like that, a **relational database** is the right tool. It stores data in **tables** (like spreadsheets) and lets you connect them. We use **PostgreSQL** ("Postgres") — the most respected open-source one — hosted by **Supabase**, which hands us a cloud Postgres plus a ready-made login system for free.



A "foreign key" is a column that must point to a real row in another table. It's how each idea knows its owner, and each job knows its idea.

Three connected tables. The database itself enforces that every idea points to a real user — you physically can't create an orphan.

Keys, the building blocks

- A **primary key** uniquely identifies each row (every table has one). We use a **UUID** — a long random id like `da272dcd-1b4d-…` — so ids are unguessable and don't leak how many records exist.
- A **foreign key** is a column that must point to a real row in another table. `idea.user_id` must equal some `profiles.id`. The database *enforces* this; you cannot create an idea for a non-existent user. That guarantee is called **referential integrity**.

Indexes: the most important performance idea

Imagine a table with 50,000 ideas and you ask "give me all ideas for user X." Without help, the database reads *all 50,000 rows* and checks each — a "full scan," slow and getting slower. An **index** is a pre-sorted lookup structure.

ANALOGY

An index is the **index at the back of a book**. To find every mention of "Redis," you don't read all 300 pages — you flip to the index and jump straight there. A database index does the same: it turns "read everything" into "jump straight to the matches." We add an index to exactly the columns we filter or sort by (like `user_id`). In Chapter 9 you'll see a real measurement where an index turns a 48-millisecond query into a 0.08-millisecond one.

Migrations: version control for your database shape

Your set of tables and columns — the **schema** — changes as the app grows. You never edit the live database by hand. Instead you write **migrations**: small, numbered, ordered scripts that each describe one change ("create this table," "add this column"). They're like Git commits, but for your database structure. We use a tool called **Alembic** and apply them with one command: `alembic upgrade head` ("bring the database up to the latest version"). The same command, run on your laptop or on production, produces the same schema — no "it works on my machine."

WHEN YOU'LL USE THIS ELSEWHERE

Almost every app needs a database. Reach for a **relational** one (Postgres, MySQL) when your data has clear relationships and you want guarantees and powerful queries — which is most of the time. Reach for a **document** database (MongoDB) when your data is loose and schema-free. Whatever you choose: index the columns you query, and always evolve the schema through migrations.

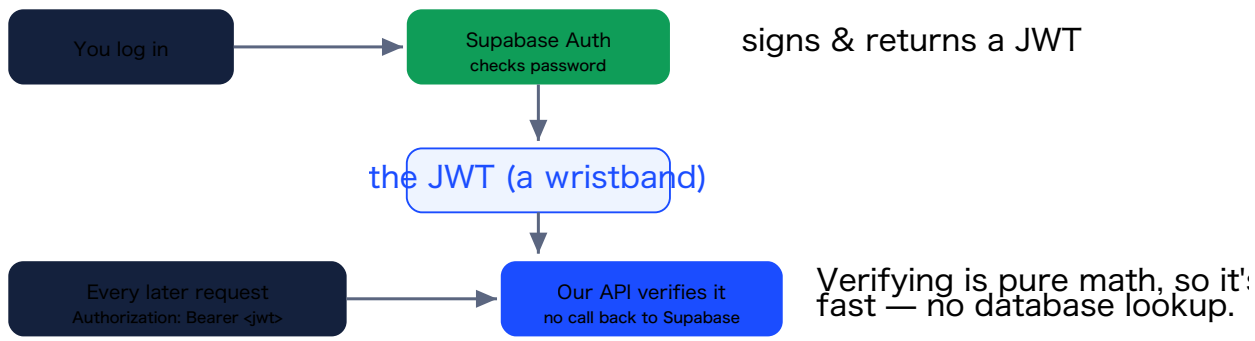
4 Authentication (How Login Really Works)

HTTP is **stateless**: every request arrives with no memory of the ones before it. So how does the server know who's making a request, without you sending your password every single time? The modern answer is a **token** — specifically a **JWT**.

What a JWT is

A **JWT** (JSON Web Token, said "jot") is a small, **digitally signed** piece of text that says, in effect: "the bearer of this is user X, issued at time Y, expiring at time Z." When you log in, the auth provider signs a token with a secret key and gives it to you. Your app sends it with every request. The server can mathematically verify the signature.

> **ANALOGY**
> A JWT is like a **festival wristband**. At the entrance you show ID once; they give you a tamper-proof wristband. After that, you flash the wristband to get into stages — nobody re-checks your ID. The wristband is hard to forge (it's signed), and security can verify it on sight without calling head office. A JWT works the same way: prove who you are once, then carry a verifiable token.



The login flow. The token is readable by anyone but tamper-proof — change one character and the signature no longer matches.

The crucial distinction: authentication vs authorization

Authentication	Authorization
"**Who are you?**"	"**Are you allowed to do *this*?**"
Done by verifying the JWT.	Done in each endpoint (e.g. "is this idea yours?").
Output: a trusted user id.	Output: allow or deny.

A valid token proves who you are; it does *not* automatically grant access to a specific idea. You need both checks. Skipping the second is one of the most common security holes in real apps.

IN STARTUPIQ — A REAL DEBUGGING STORY

We chose **Supabase Auth** so we never had to build login, password hashing, or session storage ourselves — a huge amount of security-critical work, handled for us. Getting it wired up took three rounds of debugging (a wrong signing algorithm, then a missing crypto library), all surfaced by writing a tiny script that printed the *real* error instead of a vague "invalid token." The lesson that outlives the bug: when a system gives you a generic failure, make it tell you the truth before you theorise. Debugging by inspection — not guessing — is the actual skill.

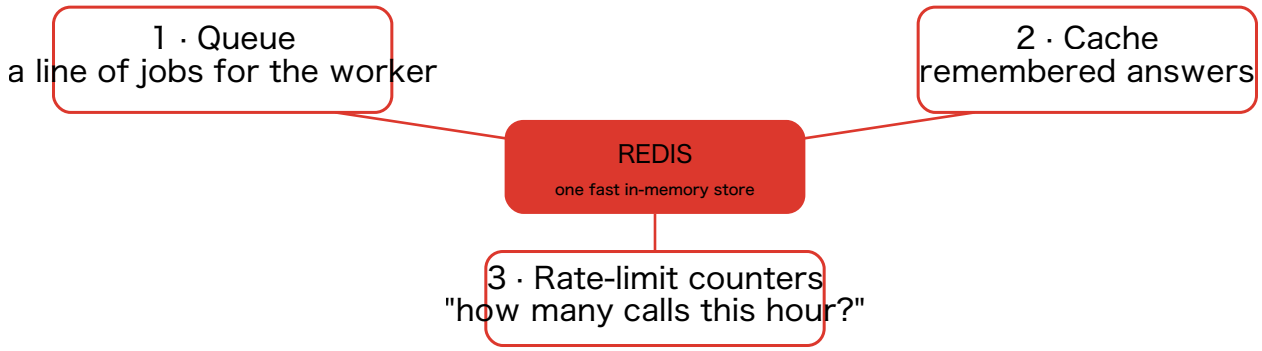
WHEN YOU'LL USE THIS ELSEWHERE

Nearly every app with users needs auth. Use a managed provider (Supabase Auth, Auth0, Firebase Auth, Clerk) unless you have a strong reason not to — rolling your own login securely is genuinely hard. Whatever you use, the JWT pattern (verify a signed token on each request, then separately check permissions) is the standard you'll see everywhere.

5 Redis: Cache, Queue & Rate Limiter

Redis is an **in-memory data store** — it keeps data in RAM (memory) instead of on disk, which makes it blazingly fast. In StartupIQ it does *three different jobs*. Learning Redis once gives you all three, which is a big part of why it's everywhere.

> **ANALOGY**
> Think of Redis as the **kitchen's whiteboard and ticket rail** next to the slow oven (your database). The chefs scribble quick notes on it, hang order tickets on it, and tally how many coffees each table has ordered — all things you want fast and don't want to walk to the back office (the database) for. Same board, many uses.



One Redis, three roles. The same store is the job queue, the cache, and the rate-limit tally.

Role 1 — Cache: don't recompute what you just computed

A **cache** stores the result of expensive work so a repeat request is instant. The pattern is **read-through**: check the cache first; on a "miss," do the real work, store it, and return it. Every request after that, until the entry expires, is a "hit" — served from Redis without touching the database.

The danger is **staleness** (the cached copy is out of date). Two defences: a **TTL** (time-to-live — every entry self-destructs after, say, 60 seconds) and **invalidation** (when the data changes, delete the cached copy so the next read recomputes). In StartupIQ, reading one idea is cached and invalidated whenever you edit it; the dashboard stats are cached with just a TTL because being a minute stale is fine.

> **ANALOGY**
> A cache is leaving the answer to a question you get asked a lot on a **sticky note on your monitor**. Anyone who asks gets the instant answer. But if the real answer changes, you must throw the note away (invalidate) — or the note will lie. "Cache invalidation" is famously one of the two hard problems in computer science.

Role 2 — Queue: a line of work for later

When the API has slow work (an AI call), it doesn't do it itself — it writes a "job" onto a **queue** in Redis. A separate **worker** program takes jobs off the front of the line and does them. We cover this fully in the next chapter; just know the queue lives in Redis.

Role 3 — Rate limiter: a budget per user

Each AI evaluation costs real money. A **rate limiter** caps how many requests a user can make in a window of time (e.g. 60 per hour) and rejects the rest with status `429 Too Many Requests`. The count is kept in Redis. Why Redis and not in the program's memory? Because when you run several copies of the API (Chapter 9–10), they must *share* one counter — otherwise each copy would grant its own budget and the limit would silently multiply. (You'll see this proven in the Kubernetes chapter.)

> **WHEN YOU'LL USE THIS ELSEWHERE**
>
> Reach for Redis whenever you need something *fast* and *shared* across your servers: caching database results or API responses, a job queue, rate limiting, session storage, real-time leaderboards, or a counter. Its speed comes from living in memory; the trade-off is that memory is smaller and more expensive than disk, so you cache what's hot, not everything.

6 Background Jobs & Workers

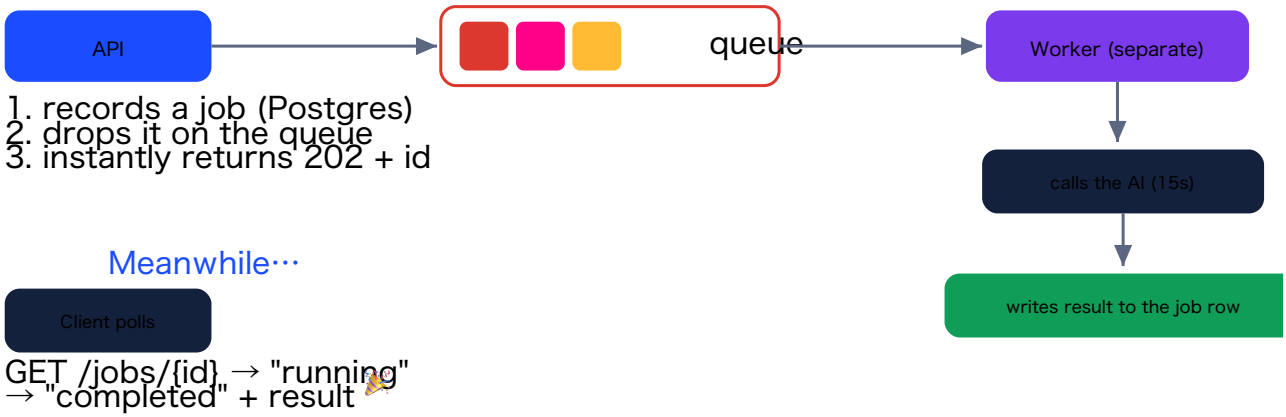
This is one of the most important patterns in all of backend engineering: **move slow work out of the request and into a background worker.** Learn it once, use it forever — for sending email, processing video, generating reports, or calling an AI.

The problem

An AI call takes 10–20 seconds. If the API does it inside the request, two bad things happen: the user's browser is frozen the whole time, and the server's limited "workers" are each tied up doing nothing but waiting — ten users could bring it to its knees. That's a scalability disaster.

The cast

- **Job** — a unit of work to do later ("run competitor research for idea X"). We also save each as a row in a `jobs` table so we have a durable record.
- **Queue** — a line jobs wait in. Producers add to the back; the worker takes from the front. Lives in Redis.
- **Worker** — a *separate program* that loops: take a job, do it, repeat. We use **Arq** (a small, Python-async job library on top of Redis).



The API just records and enqueues, then answers instantly. The worker — a different process — does the slow part where no one waits.

The status `202 Accepted` means exactly "I've accepted your request and will process it, but I'm not done." Perfect for "your job is queued." The worker runs in its own process (its own terminal, or its own container/pod later), so a flood of slow AI calls never slows down the API.

IN STARTUPIQ

Clicking "Evaluate" on a tile sends a `POST` that returns a `job_id` in milliseconds; the tile then polls every 2 seconds and shows *Queued… → Analyzing… → result*. You can fire all six tiles at once and they run in parallel — because the slow work is off the request path. To handle more evaluations at once, you simply run more worker copies.

WHEN YOU'LL USE THIS ELSEWHERE

Any time a request triggers work that's slow or can fail and retry — sending emails, resizing images, generating PDFs/reports, charging a card, calling a third-party API, video transcoding — push it to a queue and a worker. Popular tools: Arq/Celery/RQ (Python), BullMQ (Node), Sidekiq (Ruby). The shape is always the same: enqueue, work, report back.

7 Webhooks (Event-Driven Design)

So far the app only speaks when spoken to: a client asks, the server answers. A **webhook** flips that — it lets your server *proactively notify* another system when something happens, without being asked. "Your evaluation is ready." "A payment cleared." It's the reverse of a normal API call.

> **ANALOGY**
> A normal API call is **you phoning the pizza shop to ask "is it ready?"** over and over. A webhook is **giving them your number so they text you when it's out of the oven.** You register a URL; they call it when the event happens.

If the receiver is down, we retry with growing delays (2s, 4s, 8s⋯) and log every atter

A webhook delivery. Two hard problems hide here: proving it's really from us (signing) and surviving a receiver that's down (retries).

Two real problems, two standard answers

Trust — is it really from us? The receiver's URL is public; anyone could fake a "done" message. So we **sign** every payload with **HMAC**: we combine the message with a secret key (shared only with the receiver) and a hash function, producing a fingerprint sent in a header. The receiver recomputes the fingerprint over the bytes they received; if it matches, it's provably from us and untampered. This is the same keyed-signature idea as JWTs, now going outward. Stripe and GitHub sign their webhooks exactly this way.

Reliability — what if they're down? We **retry with exponential backoff** (wait 2s, then 4s, then 8s…) up to a few attempts, and record *every* attempt in an audit log so you can always see what was sent and whether it landed. Because we retry, a receiver might occasionally get the same event twice, so good receivers are **idempotent** (safe to process the same event more than once) — they dedupe on the event id.

> **WHEN YOU'LL USE THIS ELSEWHERE**
> Webhooks turn your app into an integration point. Use them to notify other systems of events ("order shipped," "build finished," "form submitted") — and on the receiving side, you'll consume webhooks from Stripe, GitHub, Slack, and countless SaaS tools. The patterns are universal: verify the signature, return a quick 200, and be idempotent.

8 Docker & Containers

You've probably heard "but it works on my machine!" That sentence is the problem Docker solves. Different computers have different operating systems, different versions of Python or Node, different libraries installed. Code that runs on your laptop can break on a server because the *environment* is different.

What a container is

A **container** packages your application *together with its entire environment* — the operating-system libraries, the language runtime, the dependencies, your code — into one sealed, runnable box. If it runs in the container on your laptop, it runs *identically* in the container on any server, because the container *is* the environment. The machine no longer matters; the environment travels with the app.

> **ANALOGY**
> A container is a **shipping container**. Before standardized containers, loading a ship meant hand-stacking barrels, sacks, and crates of every shape — slow and chaotic. The steel container changed everything: every crane, truck, and ship handles the same standard box, and it doesn't matter what's inside. Docker did this for software. Your app, whatever it's made of, goes in a standard box that any server can run the same way.

Image vs container — the one distinction to nail

Image	Container
The sealed, read-only **template**. You build it once.	A **running instance** of an image.
Like a *class* in code, or a cooking *recipe*.	Like an *object*, or the actual *meal* you cook.

One image can start many containers — which is exactly how we'll scale later (run 3 copies of the same API image).

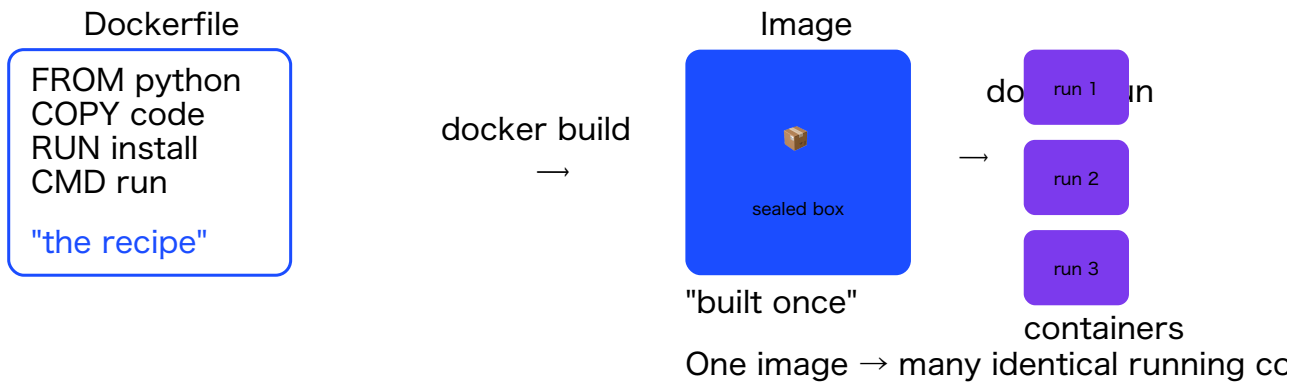
The Dockerfile: the recipe

A **Dockerfile** is a script of steps that builds an image. Here's StartupIQ's backend one, annotated:

```
FROM python:3.11-slim          # 1. start from a minimal Linux with Python 3.11
WORKDIR /app                   # 2. work inside /app
COPY pyproject.toml ./         # 3. copy the dependency list...
COPY app ./app                 #    ...and our code
RUN pip install --no-cache-dir .   # 4. install everything
EXPOSE 8000                    # 5. the app listens on port 8000
CMD ["uvicorn", "app.main:app", "--host", "0.0.0.0", "--port", "8000"]  # 6. how to run it
```

Read it as instructions to a brand-new machine: start from Python, copy our stuff in, install it, here's how to run it. Each line is a cached **layer**, so rebuilds are fast when only your code changed.

> **A CLASSIC GOTCHA**
> Inside a container, `localhost` means "only this container" — not your machine and not other containers. That's why the run command binds to `0.0.0.0` ("all network interfaces"), so other containers can reach it. This trips up everyone exactly once.

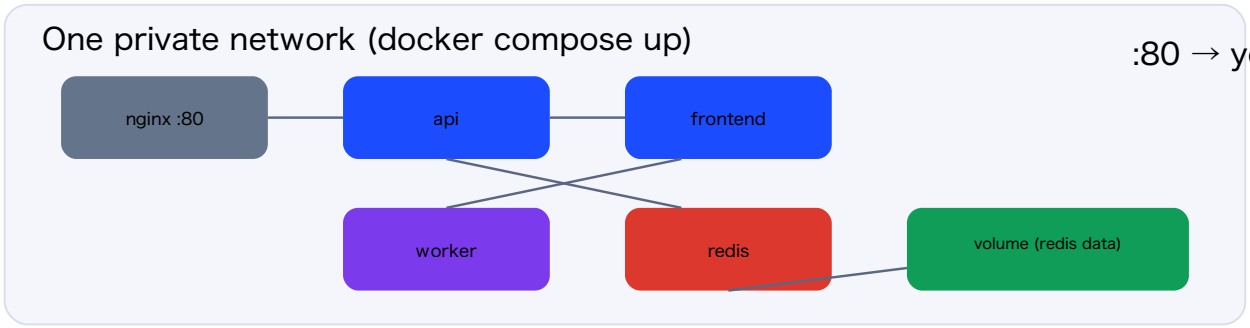


Dockerfile builds an Image; an Image runs as one or many Containers.

Docker Compose: running many containers together

One image is one container, but StartupIQ is *five* programs that must run together. **Docker Compose** describes a multi-container app in one YAML file and runs them as a unit with a single command: `docker compose up`. Three big ideas live in that file:

- **Service names become hostnames.** Compose puts all containers on a private network where each is reachable by its service name. That's why the API reaches Redis at `redis://redis:6379` — `redis` is literally the other container's name. No IP addresses, no fragile wiring.
- **Volumes persist data.** Containers are disposable; delete one and its internal files vanish. A named **volume** is storage that lives *outside* a container's life, so Redis's data survives a restart.
- **One front door.** Only the nginx container exposes a port to your machine; everything else is internal. (More on that in the next chapter.)



Five services as one app. They find each other by name; only nginx is exposed to the outside.

WHEN YOU'LL USE THIS ELSEWHERE

Use **Docker** for basically any app you want to run reliably somewhere other than your laptop. Use **Compose** to run a whole multi-service app locally with one command (it's the standard for local development of anything non-trivial). And containers are the foundation everything else stands on — the cloud and Kubernetes both run *containers*. Master this and the next chapters get much easier.

9 Load Balancers & Scaling

"Scaling" means: as more users arrive, how do you keep the app fast instead of grinding to a halt? This chapter explains the two ways to scale, the device that makes one of them possible (the load balancer), and how you *measure* whether you're actually fast.

First, how do you even know if it's fast? Two numbers

You measure with a **load test** — a tool (we used **Locust**) that simulates many users hammering the app at once. It reports two very different things:

- **Throughput** (requests per second, "RPS") — how much work the whole system gets through. Higher is better; it's your *capacity*.
- **Latency** — how long *one* request takes. But the average lies; what matters is the distribution, measured in **percentiles**:

Percentile	Means
p50 (median)	Half of requests were faster than this — the "typical" feel.
p95	95% were faster — i.e. the slowest 1 in 20. *This is what frustrated users feel.*
p99	The slowest 1 in 100 — your worst case.

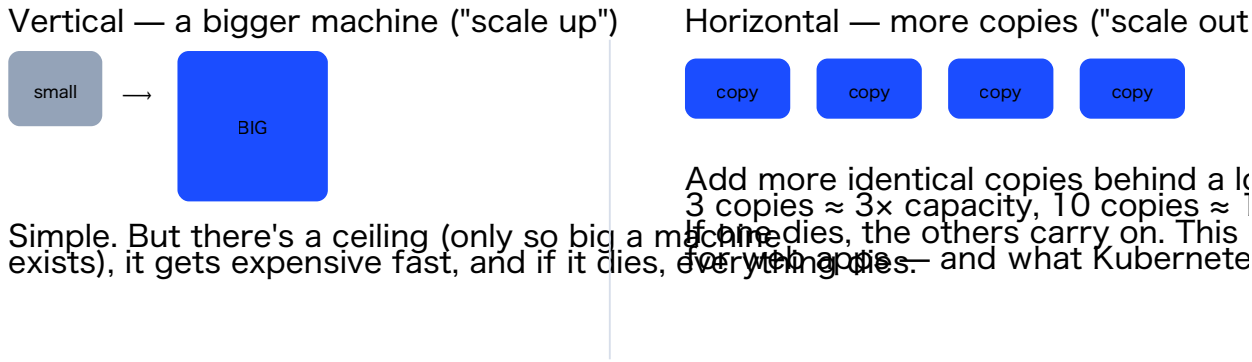
ANALOGY

Imagine a coffee shop. **Throughput** is how many coffees go out per minute. **Latency** is how long *your* coffee took. The owner brags about the average wait — but if 1 in 20 customers waits 5 minutes (a bad p95), those people are furious, no matter how good the average is. Real engineers set targets on p95/p99, never averages.

IN STARTUPIQ — A REAL RESULT

Our load test showed cached endpoints answering in **6 milliseconds** (median) while the uncached list took **410 ms** — the cache was making a ~70× difference. And the dashboard's p99 spiked to 1300 ms: that was the single cold "cache miss" (the first request, before anything was cached). The percentiles told the whole story; the average hid it.

The two ways to scale

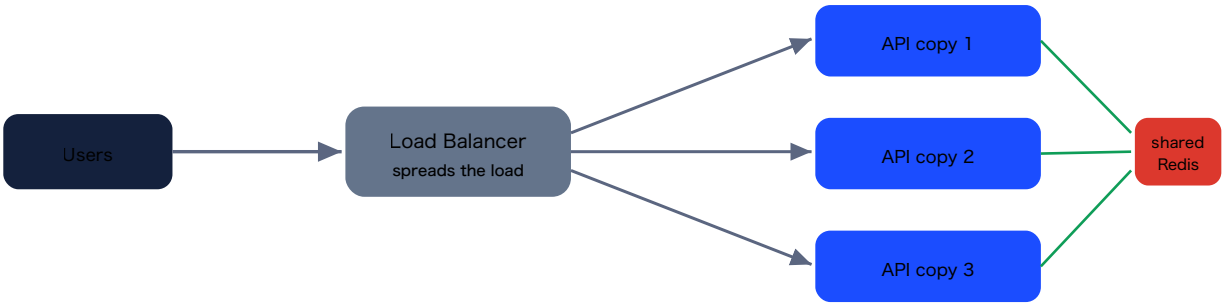


Vertical scaling = a bigger box. Horizontal scaling = more boxes. Web apps lean on horizontal.

The load balancer: the traffic cop

If you have many copies of the API, something must decide which copy each request goes to. That's a **load balancer**: a single entry point that spreads incoming requests across all the healthy copies.

> **ANALOGY**
>
> A load balancer is the **"please wait to be seated" host at a busy restaurant**. Customers form one line at the door; the host sends each party to whichever table (server) is free. Diners never pick a table themselves, and if one section closes, the host just stops seating there. The load balancer does this for requests — and it also stops sending traffic to a copy that has crashed (it checks each copy's `/health` endpoint).



One front door, three workers behind it — plus one shared Redis the copies all use (see below).

The catch that ties the whole book together: stateless copies

You can only run many interchangeable copies if every copy is **stateless** — it keeps nothing important in its own memory. Why? Imagine the rate-limit counter lived inside each API copy's

memory. With 3 copies, a user would get 3 separate budgets (one per copy), and the limit would silently triple. Chaos.

That's why, from the very beginning, StartupIQ stores all shared state — the rate-limit counters, the cache, the job queue — in **Redis**, and all data in **Postgres**. The API copies hold nothing; they all read and write the same shared Redis and database. So any copy can serve any request and they all agree. *This single design choice is what makes horizontal scaling possible* — and in the next chapter you'll watch it proven (5 successes, not 15).

> **WHEN YOU'LL USE THIS**
>
> Scale **vertically** first for quick wins (a bigger database, more RAM) — it's the least effort. Scale **horizontally** when you need real capacity or resilience for the stateless parts (your web/API servers). Always design those servers to be stateless (state in a database/cache) so horizontal scaling stays correct. And whenever you're unsure if something is fast enough — *measure* with a load test and read the p95/p99, don't guess.

10 Kubernetes

Docker Compose (Chapter 8) is great for running the stack on *one* machine. But the moment you want *many* copies of a service, automatic restarts when something crashes, updates with no downtime, and automatic scaling under load — you've outgrown Compose. That's the job of an **orchestrator**, and **Kubernetes** (often written "k8s") is the industry standard.

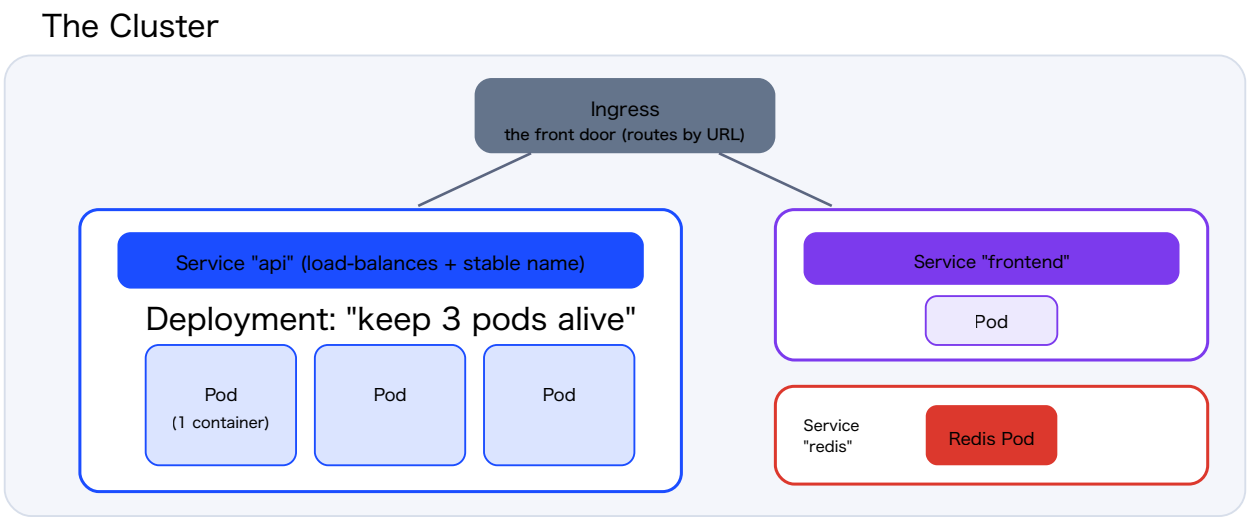
What Kubernetes is, in one breath

You give Kubernetes a *description of what you want* — "I want 3 copies of the API, 1 worker, 1 Redis, reachable at this address" — and it works **continuously to make reality match that description**. A copy crashes? It starts a new one. A machine dies? It moves the work elsewhere. You ask for 5 copies instead of 3? It creates 2 more. You stop telling it *how* and start telling it *what you want*; it handles the how, forever.

> **ANALOGY**
> Kubernetes is an **air-traffic controller for your containers**. You don't fly the planes (run containers) yourself. You declare the schedule ("3 flights to this gate"); the controller continuously directs planes, reroutes around a closed runway, and adds flights when demand spikes — keeping the whole airport matching the plan without you micromanaging each plane.

The five nouns you actually need

Kubernetes has a hundred concepts. You need five. You write each as a **YAML** file (a description) and apply it with `kubectl apply` (`kubectl` is the command-line tool for talking to a cluster).



Five nouns in one picture: Pods run containers; a Deployment keeps N pods alive; a Service is a stable name + load balancer for them; an Ingress is the front door.

Noun	What it is
Pod	The smallest unit: one running container. Roughly "one instance of one of your programs."
Deployment	"Keep N identical pods running." The workhorse. Says `replicas: 3` → always keep 3 API pods alive; replace any that die; roll out new versions gradually.
Service	A *stable address* for a set of pods *and a load balancer*. Pods come and go with changing IPs; the Service gives them one name and spreads traffic across them. (This is the load balancer from Chapter 9, built in.)
ConfigMap / Secret	Configuration injected into pods as environment variables — non-secret in a ConfigMap, sensitive (API keys) in a Secret.
Ingress	The cluster's front door — routes external traffic by URL path to the right Service (the k8s version of nginx).

Self-healing, for free

Remember that trivial `/health` endpoint from Chapter 1? Here's its payoff. You tell Kubernetes to check it:

- **Readiness probe** — don't send traffic to a pod until `/health` says 200. So during a deploy, new pods only get requests once they're truly ready.
- **Liveness probe** — if a running pod stops answering `/health`, assume it's wedged and **restart it automatically.** Your app heals itself at 3 a.m. with no one awake.

Scaling is one command — and autoscaling is automatic

```
kubectl scale deployment/api --replicas=5     # go from 3 to 5 copies, instantly
```

Kubernetes creates two more pods; the Service immediately starts load-balancing across all five. No code, no downtime. You can even let it scale *itself*: a **HorizontalPodAutoscaler** watches CPU and adds pods under load, then removes them when idle — the app right-sizes to demand.

> **IN STARTUPIQ — PROVING THE DESIGN**
>
> We ran the API as 3 pods, set the limit to 5 requests/minute, and fired 7 requests through the load-balancing front door. Result: **5 succeeded, then 429, 429.** Not 15. If each pod counted in its own memory, three pods would have allowed 5 each. Getting *5 total* proves the counter lives in *shared Redis* and the limit holds across the whole fleet. That one result is the entire payoff of building the app stateless — horizontal scaling done *correctly*.

HONEST NOTE FOR BEGINNERS

Kubernetes is powerful but heavy. The fiddly part of running it locally is always the same two things: making your built images visible to the cluster, and installing an ingress controller. The *concepts* (the five nouns) are simple and transfer everywhere; the operational details are where you'll spend time. For a small project, you often don't need Kubernetes at all — a single server with Docker Compose (next chapter, Path B) is plenty. Reach for k8s when you genuinely need many machines, auto-healing, and autoscaling.

WHEN YOU'LL USE THIS ELSEWHERE

Kubernetes is what large teams use to run lots of services across many machines reliably. You'll meet it at most mid-to-large companies. But the same manifests you write locally run on any managed cluster (Google GKE, AWS EKS, Azure AKS, DigitalOcean) — only "where's the cluster" changes. Learn the five nouns and you can read and write k8s anywhere.

11 GitHub & GitHub Actions (CI/CD)

Git and GitHub — the difference

Git is a tool on your computer that tracks the history of your code — every change, who made it, and the ability to go back in time. **GitHub** is a website that *hosts* your Git repositories in the cloud, so you can back them up, share them, and collaborate.

> **ANALOGY**
> Git is the **"track changes" + version history in a document** — every save is a labelled snapshot you can return to. GitHub is **Google Docs in the cloud** where that document lives so others can see it and work on it. Git = the tool; GitHub = the place it lives online.

The everyday commands: `git add` (stage your changes), `git commit` (save a labelled snapshot locally), `git push` (upload your commits to GitHub).

> **THE #1 RULE: NEVER COMMIT SECRETS**
> Your real API keys live in `.env` files. If you commit them to GitHub, you've published your passwords to anyone who can see the repo — a very common, very costly mistake. A `.gitignore` file lists what Git must never include (all `.env` files, `node_modules`, etc.). Always check `git status` before pushing to confirm no secret file is staged.

What CI/CD is

By itself, `git push` just *stores your code* on GitHub — nothing else. But you can attach an automated robot that runs every time you push. That robot is **CI/CD**:

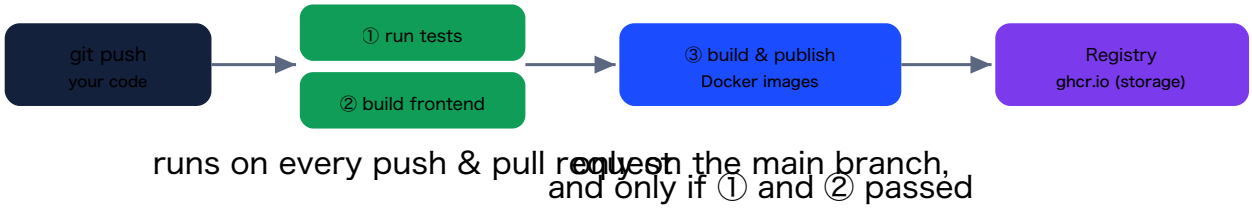
- **CI — Continuous Integration:** every push automatically *runs your tests and builds the project*, so a bug is caught within a minute, not discovered in production next week.
- **CD — Continuous Delivery:** when code lands on the main branch, it automatically *builds and publishes the deployable artifacts* (our Docker images), ready to ship.

> **ANALOGY**
> CI/CD is a **factory quality-control line** bolted to your code. Raw material (your code) goes in; it's automatically inspected (tests), assembled (build), and packaged (images) — the same way every single time, so no human forgets a step and no broken product ships.

GitHub Actions: the robot

GitHub Actions is GitHub's built-in automation. You describe a *workflow* in a YAML file at `.github/workflows/ci.yml`, and GitHub runs it on its own throwaway servers whenever you

push. (No workflow file = no automation — that's why your past projects had no checks. CI/CD is opt-in, switched on by adding this one file.)



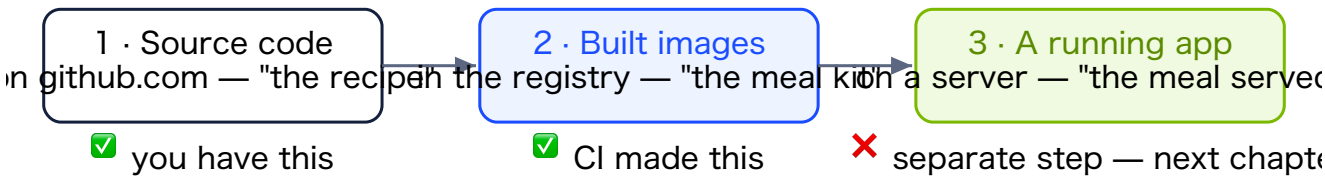
StartupIQ's pipeline. The guards matter: images only publish after tests pass, and only on main — so broken code can never ship.

A lovely payoff appears here: our tests are **hermetic** (they fake Redis, the AI, and auth), so they run on a bare server with *no secrets and no database*. A test suite that needed real infrastructure couldn't run in CI; ours can — because we designed every dependency to be swappable. The design choice we made for testability is exactly what makes automated CI possible.

> **IN STARTUPIQ — A REAL CI BUG**
> The first pipeline run failed in a way that *never* happened locally: a piece of code built a network client at import time using a config value, and in CI that value was empty (no `.env`), so it crashed before any test ran. The fix — build the client lazily, on first use — and the lesson: *don't do work that depends on configuration at import time.* This is exactly why CI is valuable: it runs your code in a clean, secret-free environment and catches assumptions your laptop was quietly hiding.

The crucial clarification: a green pipeline is NOT a deployed app

This confuses almost everyone. When the pipeline goes green, GitHub did **not** put your app online. It tested your code and *packaged* it into images sitting in storage. Nothing is *running*. Your project exists in three separate states, and only one of them ever serves users:



CI/CD gets you tested, published images (state 2). Actually running them on a server (state 3, "deploy") is a separate, optional step.

WHEN YOU'LL USE THIS

Add CI to *any* project with more than a throwaway amount of code — even solo. It's the cheapest way to never ship a broken build. GitHub Actions, GitLab CI, and CircleCI all do the same thing. Start with just "run the tests on every push"; add building and publishing later. Most of the value is in that first step.

12 Cloud Deployment: From Code to Live

The final step is taking those published images and running them on a public server so real people can use the app. The wonderful thing: **almost nothing about the code changes**, because we built for this all along.

The whole journey of a code change

Fully automated from your `git push` to a tested, published image — with one deliberate step to run it.

Why so little changes: the 12-Factor payoff

Throughout this project we followed a set of principles called the **Twelve-Factor App**. The headline one: *store configuration in the environment, not in code.* Because of that, production differs from your laptop only in **configuration**, never in code:

Service	Local	Production change
Database + Auth	Supabase (cloud)	Nothing — already cloud.
Redis	your own Redis	Point `REDIS_URL` at a managed Redis (e.g. Upstash). One env var.
The AI	OpenAI	Set `LLM_PROVIDER=claude` + the key. (This is why we hid the LLM behind one interface.)
Secrets	`.env` files	Stored in the platform's secret manager, never in git.

Two ways to actually run it

- **Path A — Managed Kubernetes** (the scalable way). Reuse the Chapter 10 manifests, but point the images at the registry and Redis at a managed service, on a real cluster (GKE/EKS/AKS/DigitalOcean). Everything you practiced — replicas, autoscaling, the rate limiter holding across pods — now on real hardware.
- **Path B — A single small VM** (the simple, cheap way). One ~$5–10/month server running Docker Compose with production config, behind a tool like Caddy that adds HTTPS automatically. Perfect for a solo project. When you outgrow it, Path A is waiting with the same images.

A REALISTIC FIRST DEPLOY

Take Path B: a small cloud VM running your Compose stack, a free managed Redis (Upstash), Claude as the AI, and Caddy for automatic HTTPS on your domain. That's a real, production-shaped deployment of everything you built, for a few dollars a month — without the operational weight of a full Kubernetes cluster.

What you actually learned

The startup evaluator was scaffolding. The durable skills are bigger: taking a vague idea, breaking it into buildable pieces, making deliberate trade-offs, debugging the inevitable surprises, and assembling something that works at scale. You now understand — and have *used* — APIs, databases, auth, caching, queues, workers, webhooks, containers, load balancing, orchestration, and CI/CD. Those transfer to any system you'll ever build.

Go build the next thing.

Appendix Glossary of Every Term

Plain-English definitions. Skim now, return when a word stops you.

Web & APIs

- **API** — a contract for how programs talk: send a request shaped like this, get a response shaped like that.
- **REST** — an API style: URLs are resources (nouns), HTTP methods are verbs.
- **HTTP method** — the verb: GET (read), POST (create), PATCH (update), DELETE (remove).
- **Status code** — the numeric result: 200 OK, 201 Created, 202 Accepted, 404 Not Found, 401 Unauthorized, 429 Too Many Requests.
- **JSON** — a simple text format of keys and values for sending data.
- **FastAPI** — the Python framework we use to build the API.
- **Endpoint** — one API door: a method + path handled by one function.
- **CORS** — a browser rule that blocks a page from calling a different origin unless allowed; a reverse proxy removes the need for it.

Data & Auth

- **Postgres / relational database** — stores data in connected tables.
- **Primary key / foreign key** — a row's unique id / a column pointing to another table's row.
- **Index** — a pre-sorted lookup that turns a slow full scan into an instant jump.
- **Migration** — a versioned script that changes the database shape (via Alembic).
- **JWT** — a small signed token proving who you are; verified by math, not a database lookup.
- **Authentication vs authorization** — "who are you?" vs "are you allowed to do this?" — both required.
- **Supabase** — hosted Postgres + a ready-made login system.

Speed, scale & events

- **Redis** — a fast in-memory store; here a queue, a cache, and a rate-limit counter.
- **Cache** — stored results of expensive work; **TTL** = expiry; **invalidation** = deleting a stale copy.
- **Queue / worker** — a line of jobs / a separate program that runs them. Slow work lives here.
- **Rate limiter** — caps requests per user per time window; returns 429 over the limit.
- **Webhook** — a URL you register so a server can POST you when an event happens (reverse of an API call).
- **HMAC** — a keyed signature proving a message is authentic and untampered.
- **Idempotent** — safe to do more than once with the same effect (important for webhook retries).
- **Throughput (RPS)** — requests/second handled. **Latency percentile (p95/p99)** — the slow tail users feel.

Infrastructure

- **Container** — an app packaged with its whole environment, so it runs identically anywhere.
- **Image vs container** — the built template vs a running instance of it.
- **Dockerfile** — the recipe that builds an image. **Docker Compose** — runs many containers as one app.
- **Volume** — storage that outlives a container, so data survives restarts.
- **Reverse proxy** — a single front door that routes by URL (nginx); makes the app one origin (no CORS).
- **Load balancer** — spreads requests across many copies of a server.
- **Vertical vs horizontal scaling** — a bigger machine vs more copies of it.
- **Stateless** — keeping no important data in a process's memory (it's in Redis/Postgres); what makes horizontal scaling correct.
- **Kubernetes (k8s)** — an orchestrator: you declare desired state, it makes reality match (scaling, healing, load-balancing).
- **Pod / Deployment / Service / Ingress** — one container / "keep N alive" / stable name + load balancer / the front door.
- **HPA** — autoscaler that adds/removes pods based on load.

Shipping

- **Git vs GitHub** — the version-history tool vs the website that hosts repositories.
- **CI/CD** — auto-run tests/builds on every push (CI) + auto-publish deployable images (CD).
- **GitHub Actions** — GitHub's automation that runs your workflow on push.
- **Container registry** — storage for built images (ghcr.io); the bridge between build and deploy.
- **12-Factor App** — principles for portable apps; chiefly "config in the environment, not in code."

— End of book —

Built alongside the StartupIQ project. Keep it next to the code.